

Sliding-Window Attention.

Full self-attention costs $O(T^2)$ — ruinous at long context. Mistral restricts each token to a local window of the previous W keys, dropping the cost to $O(T \cdot W)$, and recovers global reach through depth: stacking L layers grows the receptive field to $\approx L \cdot W$. Every number on the following pages is computed in full — nothing summarised or deferred.

THE EQUATION

$$\text{Attention}(Q, K, V)_t = \text{softmax}\left(\frac{Q_t K_{[t-W+1:t]}^\top}{\sqrt{d_k}}\right) V_{[t-W+1:t]}$$

Worked example. A banded mask with $T = 8$, $W = 3$; cost comparison at $W=4096$, $T=16384$; receptive field over $L=32$ layers.

Topic 1 of 2 in this module · companion to the course page · v1.0

Contents

1	The claim: $O(T^2) \rightarrow O(T \cdot W)$	2
2	The band mask (TikZ data flow)	3
3	Cost: full vs. sliding	3
4	Receptive field grows as $\approx L \cdot W$	4
5	Properties / consequences that matter	4
6	Where this sits in the track	4
A	Reproducible (NumPy)	5

1 The claim: $O(T^2) \rightarrow O(T \cdot W)$

In standard causal self-attention, query t attends to all keys $1, \dots, t$. The full score matrix is $T \times T$, so both compute and memory scale as $O(T^2)$. *Sliding-window attention* (SWA) keeps only a local band: query t attends to the W most recent keys, $\{t - W + 1, \dots, t\}$. The per-query work is constant ($\leq W$), so the layer costs $O(T \cdot W)$.

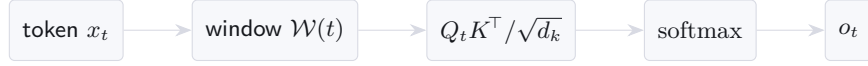
$$\text{SWA}(Q, K, V)_t = \text{softmax}\left(\frac{Q_t K_{\mathcal{W}(t)}^\top}{\sqrt{d_k}}\right) V_{\mathcal{W}(t)}, \quad \mathcal{W}(t) = \{\max(1, t - W + 1), \dots, t\} \quad (1)$$

Symbol	Meaning	Shape
Q, K, V	query / key / value projections	$T \times d_k$
T	sequence length (tokens)	scalar
W	window width (keys per query)	scalar
$\mathcal{W}(t)$	allowed key set for query t	$\leq W$ indices
d_k	head dimension	scalar

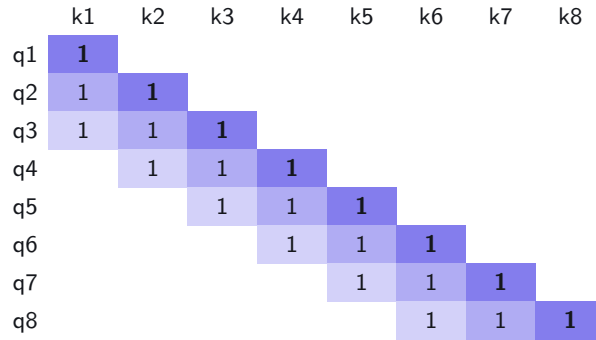
A token does not need to look at the whole document on every layer. It looks at a small recent window; the information from far away has already been *summarised into* the nearby tokens by lower layers. Reach is bought with depth, not with width.

2 The band mask (TikZ data flow)

SWA replaces the dense lower-triangular causal mask with a *banded* lower-triangular mask: keep entry (i, j) only when $i - W + 1 \leq j \leq i$. Below, the flow from a token through one SWA layer.



The allowed pattern for $T = 8$, $W = 3$ (filled = attends, intensity by how recent within the window):



Each row has at most $W = 3$ filled cells. The full causal mask would fill the entire lower triangle ($\frac{T(T+1)}{2} = 36$ entries); the band keeps only $\sum_t \min(t, W) = 1 + 2 + 3 \cdot 6 = 21$.

3 Cost: full vs. sliding

With $W=4096$ and $T=16384$ the number of score entries computed (causal, counted as T^2 for the dense upper bound) is

$$\begin{aligned} \text{full} &= T^2 = 16384^2 = 268,435,456 \ (\approx 268.4\text{M}), \\ \text{slide} &= T \cdot W = 16384 \cdot 4096 = 67,108,864 \ (\approx 67.1\text{M}), \end{aligned}$$

$$\frac{\text{full}}{\text{slide}} = \frac{T^2}{T \cdot W} = \frac{T}{W} = \frac{16384}{4096} = 4.0 \times .$$

The saving is exactly T/W : it grows linearly as the context outgrows the window.

T	full = T^2	slide = $T \cdot \min(T, W)$	ratio T/W
4,096	16,777,216	16,777,216	1.0×
8,192	67,108,864	33,554,432	2.0×
16,384	268,435,456	67,108,864	4.0×
32,768	1,073,741,824	134,217,728	8.0×
65,536	4,294,967,296	268,435,456	16.0×

(At $T \leq W$ the window covers everything, so SWA equals full attention; the gap opens only beyond $T = W = 4096$, the value used in **Mistral 7B**.)

4 Receptive field grows as $\approx L \cdot W$

A single SWA layer lets token t see back $W-1$ positions. But the value at position $t - W + 1$ already absorbed *its own* window in the layer below, so after two layers t indirectly depends on positions $\approx 2(W-1)$ back. By induction, after L stacked SWA layers the *theoretical attention span* (receptive field) is

$$R(L) \approx L(W-1) + 1 \approx L \cdot W \quad (2)$$

exactly the mechanism by which dilated/stacked convolutions enlarge their field with depth. For Mistral 7B ($W = 4096$, $L = 32$):

$$R \approx L \cdot W = 32 \cdot 4096 = \mathbf{131,072} \text{ tokens,}$$

far beyond any single layer's 4096-token window.

SWA does not throw away long-range dependencies — it *defers* them to depth. One layer reaches W ; L layers reach $\approx L \cdot W$. Cost stays $O(T \cdot W)$ per layer while the effective span scales with the number of layers.

5 Properties / consequences that matter

1. **Linear per-layer cost.** Compute and the KV-cache memory per query are $O(W)$, not $O(T)$: the rolling KV buffer holds only the last W keys/values.
2. **Equality below the window.** For $T \leq W$, $\mathcal{W}(t) = \{1, \dots, t\}$ and SWA is identical to dense causal attention — no approximation.
3. **Depth-traded reach.** Global information moves one window per layer; L layers give $\approx L \cdot W$ effective span (Eq. 2).
4. **Constant decode cost.** During generation the cache is capped at W entries, so each new token costs a fixed amount regardless of how long the sequence has grown.

Information beyond the window only propagates *through depth*. A dependency farther back than $\approx L \cdot W$ tokens simply cannot be represented, and even within that span a long-range signal must survive L hops of mixing — it is attenuated, not lost cleanly. If a task needs verbatim recall across the full context, a pure SWA stack with small W and few layers will fail.

6 Where this sits in the track

Back	Module 0.1 — dense scaled dot-product attention ($O(T^2)$, the baseline here).
This	Module 3.2 T1 — local windows trade $O(T^2) \rightarrow O(T \cdot W)$; reach via depth.
Next	Module 3.2 T2 — sparse Mixture-of-Experts routing & load balancing.

A Reproducible (NumPy)

Inputs: $W = 4096$, $T \in \{4096, \dots, 65536\}$, $L = 32$. Outputs verified above: $\text{full}(16384) = 268,435,456$; $\text{slide} = 67,108,864$; $\text{ratio} = 4.0$; $R(32) = 131,072$; banded mask ($T=8, W=3$) keeps 21 of 36 causal entries.

```
import numpy as np
W, L = 4096, 32
for T in [4096, 8192, 16384, 32768, 65536]:
    full = T*T
    slide = T*min(T, W)
    print(T, full, slide, full/slide)          # ratio == T/W (>=1)
# 16384 268435456 67108864 4.0
print("receptive field:", L*W)                # 131072

T, w = 8, 3                                  # banded causal mask
i = np.arange(T)[: , None]; j = np.arange(T)[None, :]
mask = (j <= i) & (j >= i - w + 1)
print(mask.sum(), (T*(T+1))/2)                # 21 36
```